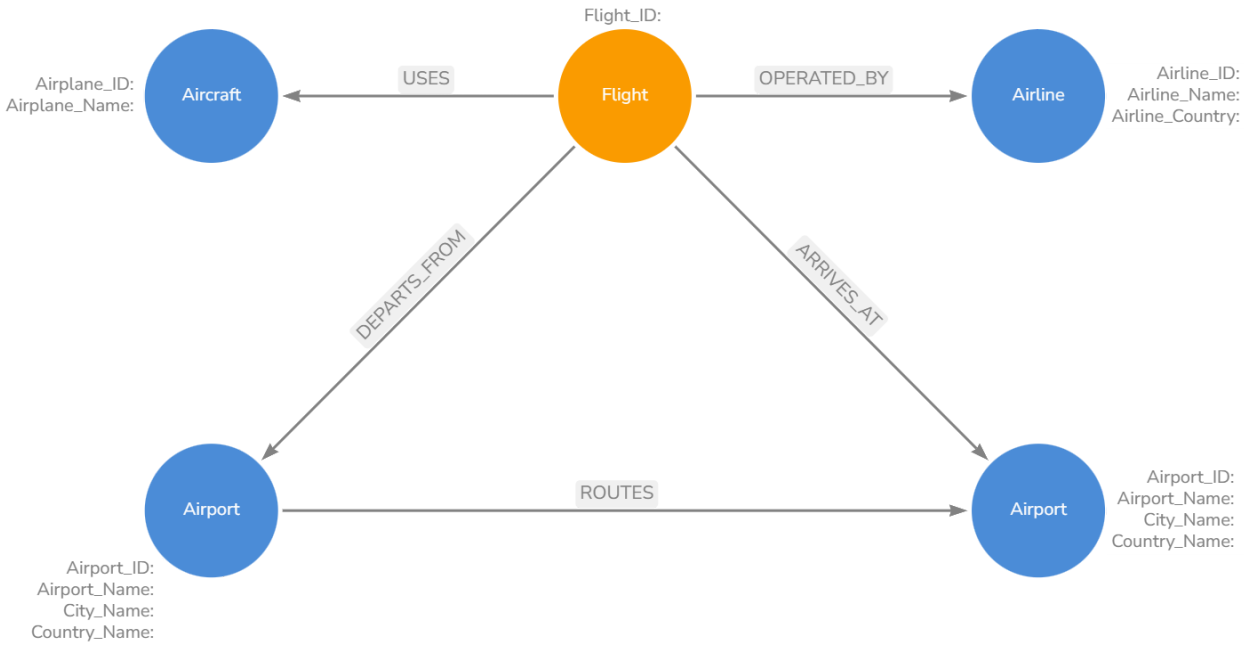
CITS5504 Project 2

Student Name	Jason Wang
Student ID	24839132

1. Graph Database Design

1.1 Arrow Apps Diagram

For this project, I designed a property graph database schema using the Arrow App tool.



1.2 Node Types and Properties

The nodes were created based on the real world entities and from the property graph model. These nodes contain specific properties that distinguish them from other nodes and will be used to answer our cypher queries.

Nodes	Properties
Airline	Airline_ID(Surrogate Key), Airline_Name
Airport	Airport_ID(Surrogate Key), Airport_Name
Aircraft	Aircraft_ID(Surrogate Key), Aircraft_Type
Flight	Flight_ID(Surrogate Key)

1.3 Relationships Between Nodes

The relationships were created to reflect logical and practical relationships among the nodes. These should be able to support intuitive traversal through nodes and efficient querying.

Relationships	From Node	To Node	Description
Departs_From	Flight	Airport	A flight departs from an airport
Arrives_At	Flight	Airport	A flight arrives at an airport
Operated_by	Flight	Airline	A flight is operated by an airline
Uses	Flight	Aircraft	A flight uses an aircraft
Routes	Airport	Airport	An airport is routed to another airport during a flight

2. Design Choice Discussion

2.1 Selection

For this project, I have first chosen 4 distinct node types to represent the key entities in the airport dataset and 4 distinct relationships to connect them.

- Airline: Represents the airline company that operates the flight route.
- Airport: Represents the airports used for the flight route. This applies to both departure and arrival airports.
- Airplane: Represents the airplane used for the flight route.
- Flight: Represents each unique flight pattern from the main dataset.
- OPERATED_BY: Connects the flight to the airline that is operating said flight.
- USES: Connects the flight to the airplane that is being used in said flight.
- ARRIVES_AT: Connects the flight to the airport that the flight arrives at.
- DEPARTS_FROM: Connects the flight to the airport that the flight departed from.
- ROUTES: Represents the pathway from one airport to another airport during a flight.

Reasons for Selecting these Nodes:

- Each node represents an actual real world entity as described in the main dataset provided.
- Separating them can provide faster, flexible and efficient querying.
- The flight node is a central node that can help distinguish each unique flight pattern, providing optimized querying performance on flight analysis.

Reasons for Selecting these Relationships:

- The relationships are directed to the flight node, this helps in identifying each unique flight pattern for more flexible and efficient querying.

- Arrival and Departure can help uniquely identify each unique airport, without the need of creating 2 different airport nodes, which might cause too many redundant rows.
- These relationships reflect real world connections between the nodes, which improves the interpretability of the graph.
- The route relationship can help with pathway findings, it's practical in a sense that airlines can use it to analyse paths for non direct flights/transfer flights.

2.2 Surrogate Keys

Surrogate keys were introduced in each node to serve as unique identifiers within the graph database. These were generated during the ETL process since the dataset did not contain any natural keys as unique identifiers.

- Airline_ID: uniquely identifies each airline.
- Airport_ID: uniquely identifies each airport.
- Airplane_ID: uniquely identifies each airplane.
- Flight_ID: uniquely identifies each flight.

The main reason to generate these keys were for 2 purposes:

- Acting as node properties for Neo4j for efficient look ups.
- Used to link the nodes when creating the relationships.

2.3 Conclusion

Pros:

- Clear representation of real world data
  The chosen nodes are directly tangible objects in the dataset, improving interpretability and making the graph intuitive and easy to understand.
- Flight-centric design provides flexible querying
  Using the flight node as a central node allows for efficient and flexible querying, since we can use it to link to each unique flight pattern.
- Reduce node redundancy by utilizing relationships
  With the use of ARRIVES_AT and DEPARTS_FROM relationships, we can avoid the duplication of creating an extra arrival airport and departure airport node.
- Scalable structure
  The model can easily incorporate more attributes without the need to redesign the graph model.
- Surrogate keys can provide reliable linking
  These ensure stable and unambiguous relationships across the graph.

Cons:

- Potential over-modeling of the flight node
  Treating each row as a unique flight node may create many nodes even when flights share identical properties and values. This can unnecessarily increase the graph size.
- Redundancy in data
  The route itself can be derived from flight airports and the relationships depart_from and arrives_at. This introduces redundancy.

3. ETL Process

3.1 ETL Goal

The main goal of the ETL process is to transform the raw dataset into a structure that is suitable to apply graph database modelling.

The ETL process mainly aims to:

- Create clean and desuplicated node tables.
- Create corresponding relationships tables in order to link the nodes based on the modelling structure discussed previously.
- Normalized the multi-values field(Plane Name)
- Generate surrogate keys to provide stable reliable identifiers for linking nodes and building relationships.
- Restructure the dataset into graph-friendly format where each row can map cleanly to a node or a relationship. This ensures efficient loading into Neo4j and creating the nodes and relationships in it.

3.2 Cleaning

We first import the pandas function and look into the first 5 rows of the dataset to get a good understanding of the dataset. This allows us to inspect the data and apply any cleaning before exporting into csv files.

```
# import pandas library, which will be used  for etl
import pandas as pd
```

```
# load CSV file into pandas DataFrames.
path = "Project2_dataset.csv"
Project_2 = pd.read_csv(path)
```

```
# use '.head()' to review tables
Project_2.head(5)
```

	Airline Name	Airline Country	Departure Airport Name	Departure Airport City	Departure Airport Country/Region	Arrival Airport Name	Arrival Airport City	Arrival Airport Country/Region	Plane Name
0	Tuna Aero	Sweden	Norsup Airport	Norsup	Vanuatu	Bauerfield International Airport	Port-vila	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
1	Tuna Aero	Sweden	Santo Pekoa International Airport	Santo	Vanuatu	Lonorore Airport	Lonorore	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
2	Tuna Aero	Sweden	Craig Cove Airport	Craig Cove	Vanuatu	Norsup Airport	Norsup	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
3	Tuna Aero	Sweden	Bauerfield International Airport	Port-vila	Vanuatu	Norsup Airport	Norsup	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
4	Tuna Aero	Sweden	Aneityum Airport	Anelghowhat	Vanuatu	Tanna Airport	Tanna	Vanuatu	Harbin Yunshuji Y12;Pilatus Britten-Norman BN-...

We can observe that the dataset does not contain any natural keys and that the Plane Name column contains multiple aircraft types stored in a single cell, separated by columns. This is not suitable for graph modelling, thus we will need to expand them into each individual node.

We continue the inspection by checking the shape, datatype and for missing values in the dataset. We can see that the dataset is fairly clean and does not have any missing values. The data type for each variable is in their correct type.

```
# check the shape of the dataset
# has over 57,300 rows and 9 variables
Project_2.shape
```

```
(57301, 9)
```

```
# check data types
Project_2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 57301 entries, 0 to 57300
Data columns (total 9 columns):
 #   Column                          Non-Null Count  Dtype 
---  ------                          --------------  ----- 
 0   Airline Name                    57301 non-null  object
 1   Airline Country                 57301 non-null  object
 2   Departure Airport Name          57301 non-null  object
 3   Departure Airport City          57301 non-null  object
 4   Departure Airport Country/Region  57301 non-null  object
 5   Arrival Airport Name            57301 non-null  object
 6   Arrival Airport City            57301 non-null  object
 7   Arrival Airport Country/Region  57301 non-null  object
 8   Plane Name                      57301 non-null  object
dtypes: object(9)
memory usage: 3.9+ MB
```

```
# check for nulls
# no nulls for now, will check again after cleaning
Project_2.isnull().sum()
```

```
Airline Name                      0
Airline Country                   0
Departure Airport Name            0
Departure Airport City            0
Departure Airport Country/Region  0
Arrival Airport Name              0
Arrival Airport City              0
Arrival Airport Country/Region    0
Plane Name                        0
dtype: int64
```

We saw that the dataset was missing natural keys and the Plane Name variable contains multiple aircraft types separated by a semi-colon. We create a surrogate key and expand the Plane Name to ensure the nodes created from the main dataset can be easily implemented into Neo4j.

We proceed first by removing duplicate values and removing empty spaces

```
# apply standard etl cleaning(removing duplicates and trimming white spaces)
Project_2.drop_duplicates()
Project_2_cleaned = Project_2.copy()
projec_2_col = Project_2.columns.tolist()
for col in projec_2_col:
    Project_2_cleaned[col] = Project_2[col].astype(str).str.strip()
Project_2_cleaned.head(5)
```

	Airline Name	Airline Country	Departure Airport Name	Departure Airport City	Departure Airport Country/Region	Arrival Airport Name	Arrival Airport City	Arrival Airport Country/Region	Plane Name
0	Tuna Aero	Sweden	Norsup Airport	Norsup	Vanuatu	Bauerfield International Airport	Port-vila	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
1	Tuna Aero	Sweden	Santo Pekoa International Airport	Santo	Vanuatu	Lonorore Airport	Lonorore	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
2	Tuna Aero	Sweden	Craig Cove Airport	Craig Cove	Vanuatu	Norsup Airport	Norsup	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
3	Tuna Aero	Sweden	Bauerfield International Airport	Port-vila	Vanuatu	Norsup Airport	Norsup	Vanuatu	Harbin Yunshuji Y12;De Havilland Canada DHC-6 ...
4	Tuna Aero	Sweden	Aneityum Airport	Anelghowhat	Vanuatu	Tanna Airport	Tanna	Vanuatu	Harbin Yunshuji Y12;Pilatus Britten-Norman BN-...

We then expand the Plane Name variable by splitting the semi-colon and stripping the empty white spaces. After, we then start loading the nodes and relationships.

```
# create a surrogate key for each row, ensures to identify each unqiue flight pattern
Project_2_cleaned['Flight_ID'] = ['FL'+str(i+1) for i in range(len(Project_2_cleaned))]
# spliting the plane name variable, by expanding each unique plane into a separate row.
Project_2_cleaned['Plane Name'] = Project_2_cleaned['Plane Name'].str.split(';')
Project_2_cleaned = Project_2_cleaned.explode('Plane Name').reset_index(drop=True)
Project_2_cleaned['Plane Name'] = Project_2_cleaned['Plane Name'].str.strip()
# to check if the separation works
Project_2_cleaned.head(5)
```

	Airline Name	Airline Country	Departure Airport Name	Departure Airport City	Departure Airport Country/Region	Arrival Airport Name	Arrival Airport City	Arrival Airport Country/Region	Plane Name	Flight_ID
0	Tuna Aero	Sweden	Norsup Airport	Norsup	Vanuatu	Bauerfield International Airport	Port-vila	Vanuatu	Harbin Yunshuji Y12	FL1
1	Tuna Aero	Sweden	Norsup Airport	Norsup	Vanuatu	Bauerfield International Airport	Port-vila	Vanuatu	De Havilland Canada DHC-6 Twin Otter	FL1
2	Tuna Aero	Sweden	Norsup Airport	Norsup	Vanuatu	Bauerfield International Airport	Port-vila	Vanuatu	Pilatus Britten-Norman BN-2A/B Islander	FL1
3	Tuna Aero	Sweden	Santo Pekoa International Airport	Santo	Vanuatu	Lonorore Airport	Lonorore	Vanuatu	Harbin Yunshuji Y12	FL2
4	Tuna Aero	Sweden	Santo Pekoa International Airport	Santo	Vanuatu	Lonorore Airport	Lonorore	Vanuatu	De Havilland Canada DHC-6 Twin Otter	FL2

3.3 Nodes

- Airline Node: The airline node was created by selecting specific fields unique for the node. The cleaning process used was to remove duplicate values.

```
# remove duplicates and keep unique records of airlines
airline_df = Project_2_cleaned.drop_duplicates(subset=['Airline Name', 'Airline Country']).reset_index(drop=True)
# add surrogate key
airline_df['Airline_ID'] = ['AL'+str(i+1) for i in range(len(airline_df))]
# select preserved fields
airline_df = airline_df[[
    'Airline_ID',
    'Airline Name',
    'Airline Country'
]]
airline_df.head(5)
```

	Airline_ID	Airline Name	Airline Country
0	AL1	Tuna Aero	Sweden
1	AL2	Cranwell Flying Training Unit	United Kingdom
2	AL3	Renan	Moldova
3	AL4	Voar Lda	Angola
4	AL5	Southern Cargo Air Lines	Russia

- Airport Node: The airport node was created by selecting the fields related to airport data. The cleaning process used was to remove duplicate values.

```
# combining both ariport columns and drop duplicate keys to only keep unique airlines
dep_airports = Project_2_cleaned[['Departure Airport Name', 'Departure Airport City', 'Departure Airport Country/Region']]
dep_airports.columns = ['Airport Name', 'Airport City', 'Airport Country/Region']
arr_airports = Project_2_cleaned[['Arrival Airport Name', 'Arrival Airport City', 'Arrival Airport Country/Region']]
arr_airports.columns = ['Airport Name', 'Airport City', 'Airport Country/Region']
airport_df = pd.concat([arr_airports, dep_airports]).drop_duplicates().reset_index(drop=True)
# add surrogate key
airport_df['Airport_ID'] = ['AP'+str(i+1) for i in range(len(airport_df))]
# select preserved fields
airport_df = airport_df[[
    'Airport_ID',
    'Airport Name',
    'Airport City',
    'Airport Country/Region'
]]
# check if airport correctly populated
airport_df.head(5)
```

	Airport_ID	Airport Name	Airport City	Airport Country/Region
0	AP1	Bauerfield International Airport	Port-vila	Vanuatu
1	AP2	Lonorore Airport	Lonorore	Vanuatu
2	AP3	Norsup Airport	Norsup	Vanuatu
3	AP4	Tanna Airport	Tanna	Vanuatu
4	AP5	Santo Pekoa International Airport	Santo	Vanuatu

- Airplane Node: The airplane node was created by selecting the Plane Name field and surrogate key was populated based on that. The cleaning process used was to remove duplicate values.

```
# combining both airplane columns
airplane_df = Project_2_cleaned[['Plane Name']]
#remove duplicates
airplane_df = airplane_df.drop_duplicates().reset_index(drop=True)
# add surrogate key
airplane_df['Airplane_ID'] = ['AR'+str(i+1) for i in range(len(airplane_df))]
# select preserved fields
airplane_df = airplane_df[[
    'Airplane_ID',
    'Plane Name'
]]
# check if airplane correctly populated
airplane_df.head(5)
```

	Airplane_ID	Plane Name
0	AR1	Harbin Yunshuji Y12
1	AR2	De Havilland Canada DHC-6 Twin Otter
2	AR3	Pilatus Britten-Norman BN-2A/B Islander
3	AR4	Yakovlev Yak-40
4	AR5	Yakovlev Yak-42

- Flight Node: After creating the relationships, we extract only the surrogate Flight_ID for the flight node and remove duplicates. This is so we can get the unique flight patterns.

```
# populate the flight node
flight_df = Project_2_cleaned[['Flight_ID']].drop_duplicates().reset_index(drop=True)
flight_df.head(5)
```

	Flight_ID
0	FL1
1	FL2
2	FL3
3	FL4
4	FL5

The Flight dataframe was created by combining each unique airline, departure airport, arrival airport and plane name. The cleaning process used was to remove duplicate values. This is mostly used to create the relationships between to connect the nodes, the flight node will only contain surrogate keys to signify the unique rows.

3.3 Relationships

To represent the association between the different nodes in the graph database, four relationships were created in order to connect them. These nodes are rel_operated_by, rel_depart_from, rel_arrives_at and rel_uses.

Each relationship was created by linking each corresponding surrogate key between relevant nodes.

- rel_operated_by: To form the relationship between Flight and Airline, I used the Flight_ID and Airline_ID to link them and name it as OPERATED_BY. The idea behind this is that a flight is operated by an airline.

```
# relationship: route OPERATED_BY airline
rel_operated_by = flight_df.merge(airline_df, on='Airline Name')[['Flight_ID', 'Airline_ID']].drop_duplicates().reset_index(drop=True)
rel_operated_by.head(5)
```

	Flight_ID	Airline_ID
0	FL1	AL380
1	FL2	AL380
2	FL3	AL380
3	FL4	AL380
4	FL5	AL380

- rel_departs_from: To form the relationship between flights and the departing airport, I used the Flight_ID and Airport_ID to link them and name it as DEPARTS_FROM. The idea behind this is that a flight departs from an airport.

```
# relationship: flight DEPARTS_FROM airline
# merge by extracting the flight id and airport id
rel_departs_from = Project_2_cleaned.merge(airport_df,
                                           left_on=['Departure Airport Name', 'Departure Airport City', 'Departure Airport Country/Region'],
                                           right_on=['Airport Name', 'Airport City', 'Airport Country/Region'])[['Flight_ID', 'Airport_ID']].drop_d
rel_departs_from.head(5)
```

	Flight_ID	Airport_ID
0	FL1	AP3
1	FL6191	AP3
2	FL6192	AP3
3	FL6346	AP3
4	FL6347	AP3

- rel_arrives_at: To form the relationship between flights and the arriving airport, I used the Flight_ID and Airport_ID to link them and name it as ARRIVES_AT. The idea behind this is that a flight arrives at an airport.

```
# relationship: flight ARRIVES_AT airport
# merge by extracting the flight id and airport id
rel_arrives_at = Project_2_cleaned.merge(airport_df,
                                         left_on=['Arrival Airport Name', 'Arrival Airport City', 'Arrival Airport Country/Region'],
                                         right_on=['Airport Name', 'Airport City', 'Airport Country/Region'])[['Flight_ID', 'Airport_ID']].drop_d
rel_arrives_at.head(5)
```

	Flight_ID	Airport_ID
0	FL1	AP1
1	FL12	AP1
2	FL16	AP1
3	FL20	AP1
4	FL6337	AP1

- rel_uses: To form the relationship between flights and the airplane it uses, I used the Flight_ID and Airplane_ID to link them and name it as USES. The idea behind this is that a flight uses an airplane.

```
# relationship: flight USES an airplane
# merge by extracting flight id and plane id
rel_uses = Project_2_cleaned.merge(airplane_df, on = 'Plane Name')[['Flight_ID', 'Airplane_ID']].drop_duplicates().reset_index(drop=True)
rel_uses.head(5)
```

	Flight_ID	Airplane_ID
0	FL1	AR1
1	FL2	AR1
2	FL3	AR1
3	FL4	AR1
4	FL5	AR1

- rel_route: To form the relationship between 2 airports and their routes, I used the Airport_ID of the departure and arrival airports to link them and name it as ROUTES. The idea behind this is that a flight uses an airplane.

```
# relationship: one airport to another airport is a route
# merge by extracting airport id from one airport to another airport
rel_routes = rel_departs_from.merge(rel_arrives_at, on = 'Flight_ID')
rel_routes = rel_routes[['Airport_ID_x', 'Airport_ID_y']].rename(columns={'Airport_ID_x': 'A1_ID', 'Airport_ID_y': 'A2_ID'})
rel_routes.head(5)
```

	A1_ID	A2_ID
0	AP3	AP1
1	AP3	AP13
2	AP3	AP5
3	AP3	AP1255
4	AP3	AP1256

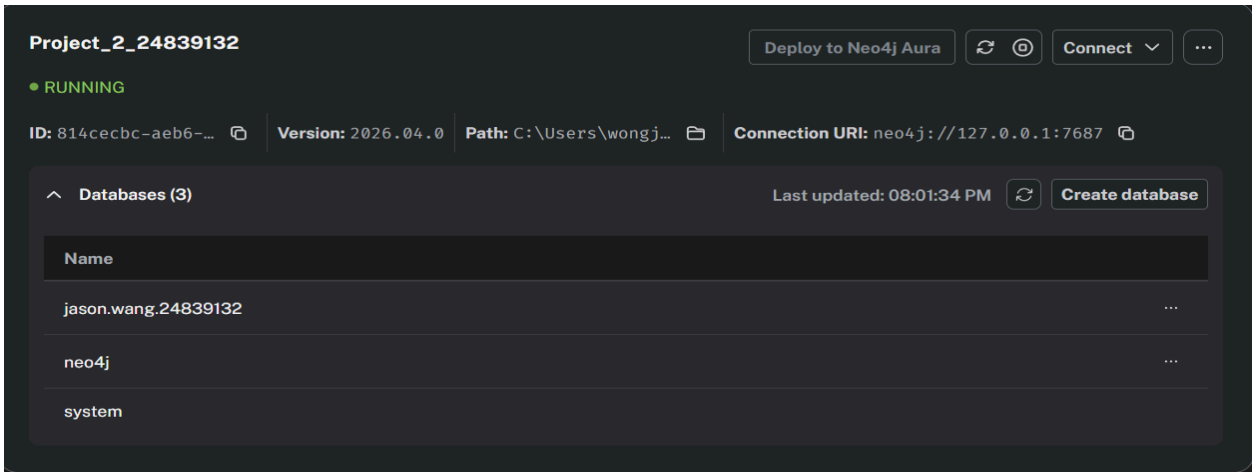
We then export the nodes and relationships into .csv files to be used to import into Neo4j, so we can create the graph database.

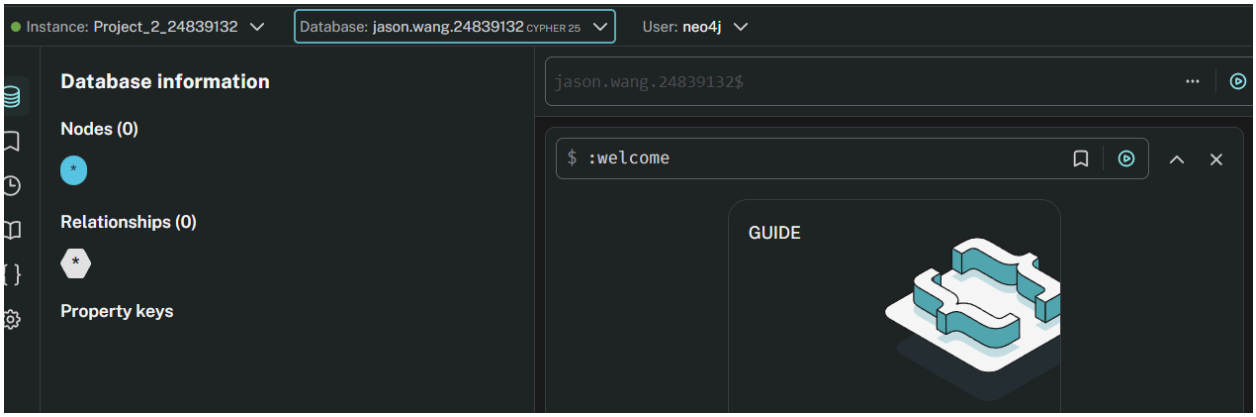
```
# export all the nodes and relationships
airline_df.to_csv('airline_node.csv', index = False)
airport_df.to_csv('airport_node.csv', index = False)
airplane_df.to_csv('airplane_node.csv', index = False)
flight_df.to_csv('flight_node.csv', index = False)
rel_operated_by.to_csv('rel_operated_by.csv', index = False)
rel_departs_from.to_csv('rel_departs_from.csv', index = False)
rel_arrives_at.to_csv('rel_arrives_at.csv', index = False)
rel_uses.to_csv('rel_uses.csv', index = False)
rel_routes.to_csv('rel_routes.csv', index = False)
```

4. Graph Database Implementation

4.1 Database Setup

A project and a database instance were created in Neo4j, since only one database could be actively connected at a time within a single database instance. The nodes and relationship csv will be imported into the same database.

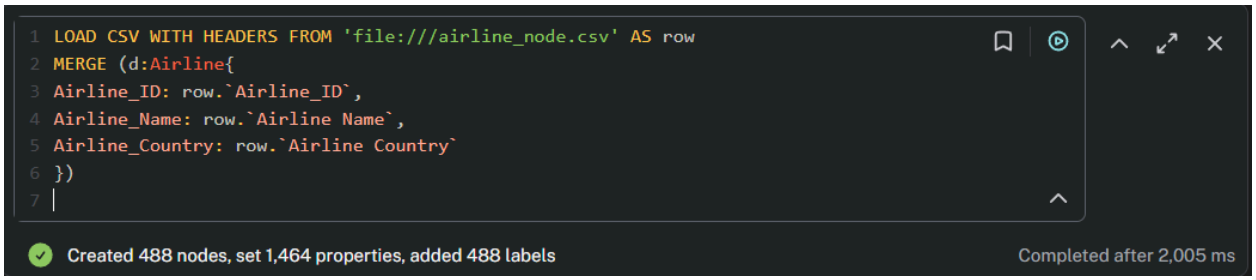




4.2 Data Import

The nodes and relationships were imported into Neo4j using the LOAD CSV WITH HEADERS FROM command statement.

Import for Airline Node:



Import for Airport Node:

```
LOAD CSV WITH HEADERS FROM 'file:///airport_node.csv' AS row
MERGE (d:Airport {
Airport_ID: row.`Airport_ID`,
Airport_Name: row.`Airport Name`,
Airport_City: row.`Airport City`,
Airport_Country: row.`Airport Country/Region`
})
```

Created 2,836 nodes, set 11,344 properties, added 2,836 labels — Completed after 4,928 ms

Import for Airplane Node:

```
LOAD CSV WITH HEADERS FROM 'file:///airplane_node.csv' AS row
MERGE (d:Airplane {
Airplane_ID: row.`Airplane_ID`,
Plane_Name: row.`Plane Name`
})
```

Created 114 nodes, set 228 properties, added 114 labels — Completed after 216 ms

Import for Flight Node:

```
LOAD CSV WITH HEADERS FROM 'file:///flight_node.csv' AS row
CREATE (d:Flight {
Flight_ID: row.`Flight_ID`
})
```

Created 57,301 nodes, set 57,301 properties, added 57,301 labels — Completed after 1,742 ms

Originally, I was importing the relationships the same way as the nodes, but the processing time was taking too long. I was able to find work around by creating indexes on the nodes and it makes importing relationships a lot faster and more efficient.

```
CREATE INDEX FOR (f:Flight) ON (f.Flight_ID);
CREATE INDEX FOR (al:Airline) ON (al.Airline_ID);
CREATE INDEX FOR (ai:Airport) ON (ai.Airport_ID);
CREATE INDEX FOR (ap:Airplane) ON (ap.Airplane_ID);
```

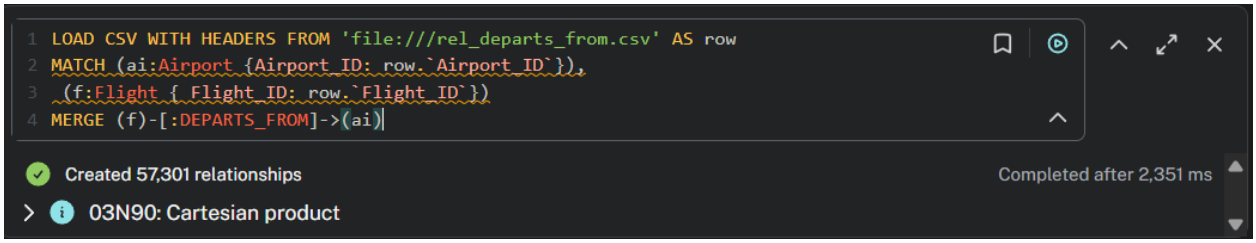
Importing OPERATED_BY relationships:

```
LOAD CSV WITH HEADERS FROM 'file:///rel_operated_by.csv' AS row
MATCH (al:Airline {Airline_ID: row.`Airline_ID`}),
 (f:Flight { Flight_ID: row.Flight_ID})
MERGE (f)-[:OPERATED_BY]->(al)
```

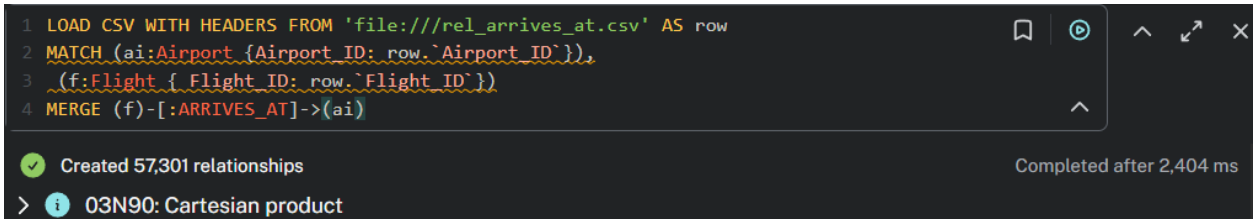
Created 57,301 relationships — Completed after 2,639 ms

03N90: Cartesian product

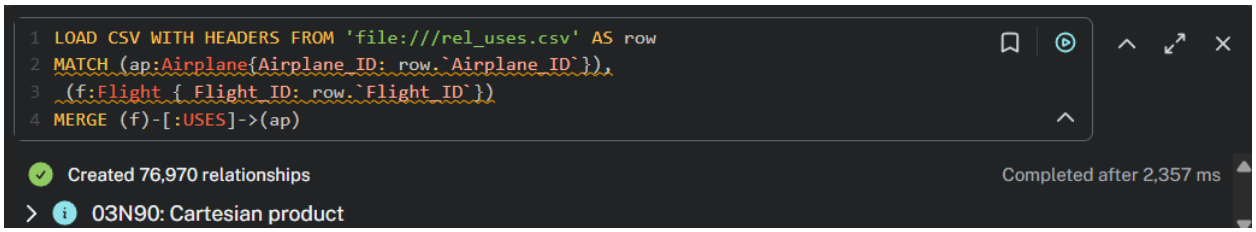
Importing DEPARTS_FROM relationships:



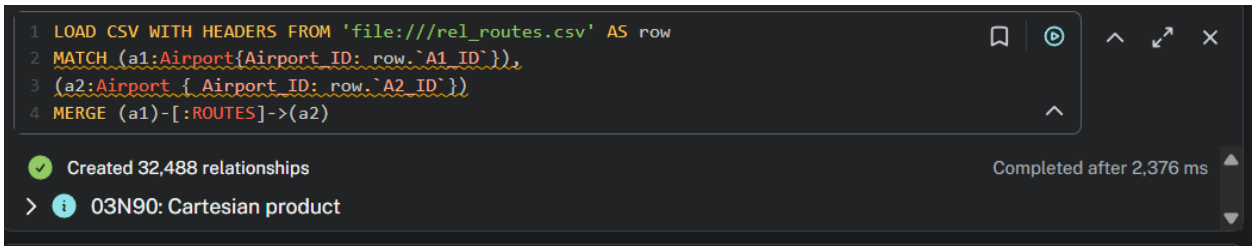
Importing ARRIVES_AT relationships:



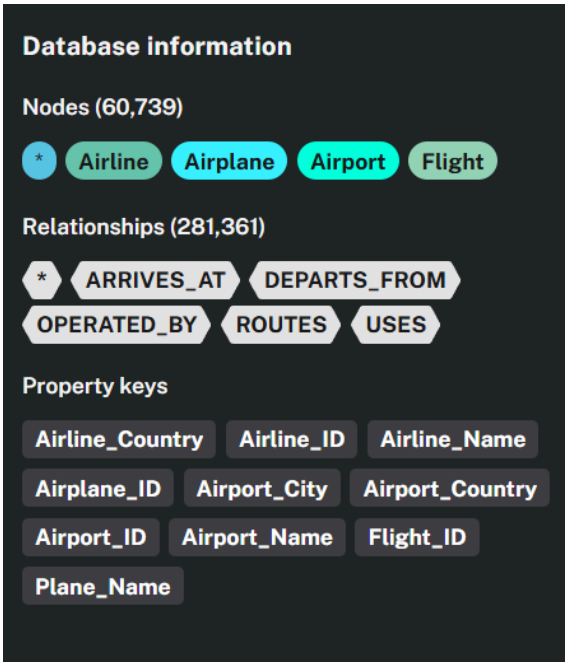
Importing USES relationships:



Importing ROUTES relationships:

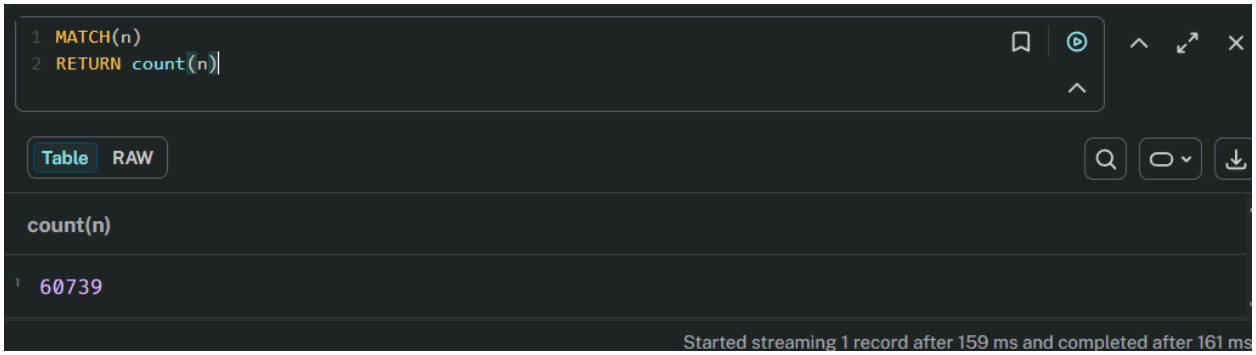


We are able to see that all the nodes, relationships and properties have been imported correctly into Neo4j.



4.3 Validation and Checking

After populating the nodes and the relationships. I first check if the nodes have been correctly populated with its proper label. Applying cypher query functions, I was able to find that there are a total of 60739 nodes and each node that was modeled from the property graph has been correctly populated.



It can be seen that flight nodes have 57301 labels, which matches with the number of rows from the original dataset. Also, the rest have a reduced number of nodes, this indicates that there were repeating values that we've removed and retained only the unique ones.

```
MATCH(n)
RETURN labels(n), count(n)
```

Table RAW

labels(n)	count(n)
["Airline"]	488
["Airport"]	2836
["Airplane"]	114
["Flight"]	57301

I then check the relationships to ensure they have been properly populated. I checked the number of relationships, and it can be seen that there are 286,563 total relationships in the graph database and each relationship has been properly populated.

```
MATCH ()-[r]->()
RETURN count(r)
```

Table RAW

count(r)
281361

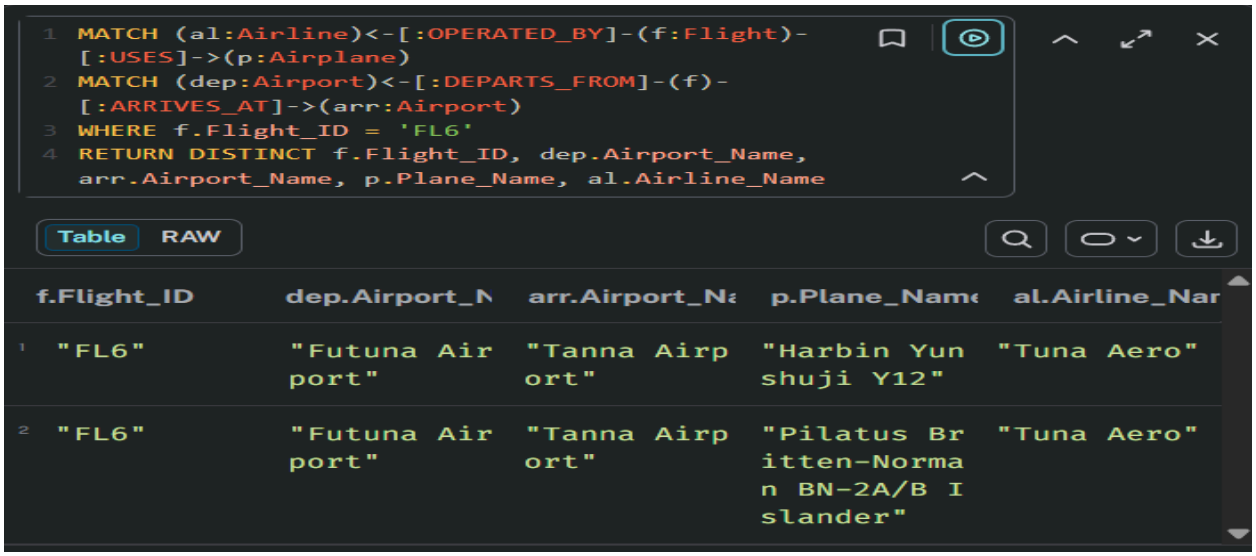
```
MATCH ()-[r]->()
RETURN type (r), count(r)
```

Table RAW

type (r)	count(r)
"ROUTES"	32488
"DEPARTS_FROM"	57301
"ARRIVES_AT"	57301
"USES"	76970
"OPERATED_BY"	57301

Started streaming 5 records after 475 ms and completed after 998 ms.

Further checking, I pulled one specific flight and checked if the nodes and relationships are correctly linked with each other. I pulled a flight whose Flight_ID is ‘FL6’. It can be confirmed that the relationships between the nodes have been correctly connected, as we can pull the departure airport, arrival airport, airline that operates the flight and the planes used for that specific flight.



```
MATCH (al:Airline)<-[:OPERATED_BY]-(f:Flight)-
[:USES]->(p:Airplane)
MATCH (dep:Airport)<-[:DEPARTS_FROM]-(f)-
[:ARRIVES_AT]->(arr:Airport)
WHERE f.Flight_ID = 'FL6'
RETURN DISTINCT f.Flight_ID, dep.Airport_Name,
arr.Airport_Name, p.Plane_Name, al.Airline_Name
```

Table RAW

	f.Flight_ID	dep.Airport_N	arr.Airport_Na	p.Plane_Name	al.Airline_Nar
1	"FL6"	"Futuna Airport"	"Tanna Airport"	"Harbin Yunshuji Y12"	"Tuna Aero"
2	"FL6"	"Futuna Airport"	"Tanna Airport"	"Pilatus Britten-Norman BN-2A/B Islander"	"Tuna Aero"

5. Cypher Queries

5.1 Given Questions

a. List all distinct airline names where the airline's country is Australia.



```
MATCH (a:Airline{Airline_Country:'Australia'})
RETURN DISTINCT a.Airline_Name
```

Table RAW

	a.Airline_Name
1	"Whyalla Airlines"
2	"Transpac Express"
3	"Transaustralian Air Express"

Started streaming 3 records after 285 ms and completed after 309 ms.

Based on the Cypher query, all the distinct Australian airlines in the dataset are: Transaustralian Air Express, Transpac Express and Whyalla Airlines.

b. How many route records in the dataset are domestic, and how many are international? A route is domestic if the departure and arrival airports are in the same country/region, and international if they are in different countries/regions.

```
MATCH (dep:Airport)<-[:DEPARTS_FROM]-(f:Flight)-[:ARRIVES_AT]->(arr:Airport)
WITH DISTINCT f,
CASE
WHEN arr.Airport_Country = dep.Airport_Country THEN 'Domestic'
ELSE 'International'
END AS route_type
RETURN DISTINCT route_type, count(f) AS total
```

Table RAW

route_type	total
"Domestic"	26146
"International"	31155

Started streaming 2 records after 338 ms and completed after 788 ms.

Based on the Cypher query, the dataset contains 26,513 domestic flights and 31,515 international flights.

c. In the given dataset, each row represents a service operated by a specific airline between two airports. Find the airport pair with the greatest number of such records. Treat A→B and B→A as the same airport pair. Use WITH in your query.

```
MATCH (a1:Airport)-[:ROUTES]-(a2:Airport)
MATCH (a1)<-[:DEPARTS_FROM]-(f:Flight)-[:ARRIVES_AT]->(a2)
WHERE a1.Airport_ID < a2.Airport_ID
WITH a1.Airport_Name AS AirportA, a2.Airport_Name AS AirportB, count(f) AS
total_records
ORDER BY total_records DESC
RETURN AirportA, AirportB, total_records
LIMIT 1
```

Table RAW

AirportA	AirportB	total_records
"Hartsfield Jackson Atlanta International Airport"	"Chicago O'Hare International Airport"	36

The airport pair with the greatest number of records is Hartsfield Jackson Atlanta International Airport and Chicago O'Hare International Airport, with a flight record of 36.

d. Each route may be operated using one or more aircraft types (stored as a semicolon-separated list in the dataset). Find the top 5 airport pairs that are served with the greatest number of distinct aircraft types across all rows and all airlines. Treat A→B and B→A as the same airport pair, and count distinct aircraft types across all rows for that pair.

```
MATCH (a1:Airport)-[:ROUTES]-(a2:Airport)
MATCH (a1)<-[:DEPARTS_FROM]-(f:Flight)-[:ARRIVES_AT]->(a2)
MATCH (f)-[:USES]->(ap:Airplane)
WHERE a1.Airport_ID < a2.Airport_ID
WITH a1.Airport_Name AS AirportA, a2.Airport_Name AS AirportB, count(DISTINCT
ap.Airplane_ID) AS Airplanes
ORDER BY Airplanes DESC
RETURN AirportA, AirportB, Airplanes
LIMIT 5
```

	AirportA	AirportB	Airplanes
1	"Incheon International Airport"	"Hong Kong International Airport"	15
2	"Hong Kong International Airport"	"Suvarnabhumi Airport"	13
3	"Shanghai Hongqiao International Airport"	"Beijing Capital International Airport"	12
4	"Guangzhou Baiyun International Airport"	"Beijing Capital International Airport"	12
5	"Incheon International Airport"	"Kansai International Airport"	12

e. There are no direct flights between Beijing (Beijing Capital International Airport) and Perth (Perth International Airport). Find all possible travel routes from Beijing to Perth where at most 3 hops (i.e., at most 3 ROUTE relationships) are traversed. Treat each unique sequence of airports visited as a distinct route. How many such distinct routes exist?

```
MATCH path=(a1:Airport{Airport_Name: 'Beijing Capital International Airport'})-
[:ROUTES*1..3]->(a2:Airport{Airport_Name: 'Perth International Airport'})
RETURN COUNT(DISTINCT path)
```

Table RAW

	COUNT(DISTINCT path)
1	599

There are 599 distinct routes from Beijing Capital International Airport to Perth International Airport with a max of 3 hops.

f. Find the top 5 pairs of airlines that compete head-to-head on the greatest number of shared routes. Two airlines are considered competitors on a route if they both operate between the same two airports, regardless of direction (treat A→B and B→A as the same route). Return the airline pair names and the number of routes they share.

```
MATCH (a1:Airport)-[:ROUTES]-(a2:Airport)
MATCH (a1)<-[:DEPARTS_FROM]-(f1:Flight)-[:ARRIVES_AT]->(a2)
MATCH (a1)<-[:DEPARTS_FROM]-(f2:Flight)-[:ARRIVES_AT]->(a2)
MATCH (al1)<-[:OPERATED_BY]-(f1)
MATCH (al2)<-[:OPERATED_BY]-(f2)
WHERE al1.Airline_ID<al2.Airline_ID AND a1.Airport_ID<a2.Airport_ID
WITH al1.Airline_Name AS AirlineA, al2.Airline_Name AS AirlineB,count(DISTINCT
[a1.Airport_ID, a2.Airport_ID]) AS Shared_Routes
ORDER BY Shared_Routes DESC
RETURN AirlineA, AirlineB, Shared_Routes
LIMIT 5
```

	AirlineA	AirlineB	Shared_Routes
1	"SAM Colombia"	"Zantop International Airlines"	804
2	"AtlasGlobal Ukraine"	"Sheremetyevo-Cargo"	358
3	"Servicos De Alquiler"	"Sheremetyevo-Cargo"	244
4	"Servicos De Alquiler"	"AtlasGlobal Ukraine"	199
5	"Sunshine Air Tours"	"Grand China Air"	192

Started streaming 5 records after 1,589 ms and completed after 40,359 ms.

5.2 Selfmade Queries

a. What are the top 5 airplanes with the most routes?

```
MATCH (f:Flight)-[:USES]->(p:Airplane)
WITH p, count(f) AS total_routes
ORDER BY total_routes DESC
RETURN p.Plane_Name, total_routes
LIMIT 5
```

Table RAW

	p.Plane_Name	total_routes
1	"Airbus A320"	15381
2	"Boeing 737-800"	10209
3	"Airbus A319"	7835
4	"Airbus A321"	3607
5	"Boeing 737"	2750

b. How many flights are connected to Perth International Airport (Applying APOC)?

```
MATCH (a:Airport{Airport_Name: "Perth
International Airport"})
RETURN apoc.node.degree(a,
"DEPARTS_FROM|ARRIVES_AT") AS flights
```

Table RAW

	flights
1	160

6. Graph Data Science Applications

6.1 What is Graph Data Science

Graph data science is a science based approach of extracting information from a graph model's interactions and properties, usually to make predictions. It is defined as a collection of strategies that help data scientists use graph data to answer questions and clarify outcomes.[1]

Graph data science works by modeling the database by using nodes and relationships rather than flat tables, such as relational databases. From this, it starts analyzing the structure of the connections among the nodes and relationships by applying some specialized algorithms to uncover discerning patterns, influential nodes, detecting clusters or anomalies, and measuring similarities between nodes.[2]

6.2 Practical Application and Algorithms

There are multiple uses of Graph data science across multiple sectors, particularly in domains where relationships among entities play a central role. By modelling interconnected nodes and relationships, it enables analysis of complex networks that tabular structures cannot provide. In addition, graph algorithms play a crucial role in analysing the overall graph structure, with the outputs being used either for standalone analysis or as a feature in machine learning models.[3]

Here are some more applications of graph data science in some domains:

Social Media

Social media such as Facebook, Twitter and Instagram apply graph data science to analyze the interactions of users and their network structures to determine the communities users interact more often, recognise the more influential users and can help predict user behavior. [4]

A specific algorithm used can be the Louvian algorithm to determine communities users interact with more often. Louvian algorithm detects communities in a large network, it maximizes a modularity score for each community, where this modularity quantifies the quality of an assignment of nodes to communities. This evaluates how densely connected the nodes within a community are compared to how connected they would be in a random network. [5]

Cybersecurity

Another application is in cyber security, since it applies graph data science to check users, IP addresses, devices and transactions and detect any anomalies or malicious activities. [4]

A specific algorithm that can be used in this setting is link prediction, a common machine learning task applied to graphs. It starts by training a model to learn between pairs of nodes in a graph, where relationships should exist. More precisely, the input into this algorithm are examples of node pairs [6]. This can be used to detect suspicious activities not yet observed.

6.3 Practical Application and Algorithms For Project

In the context of the project, the dataset can be modelled into a transportation network, given that each row shows an airline operating a flight using airplanes to depart from one departure

airport and arriving at a different airport.This graph based model allows for complex queries about connectivity, routes and operational patterns to be performed efficiently.

Some algorithms we can apply are:

- Degree Centrality: We can use this to identify important and highly connected airports within the network. This metric measures the value of a node in a graph depending on its degree, a higher degree means this node is more important in a graph [3]. Airports with high degree centrality are considered major hubs with significant connections and high foot traffic, making them significant in understanding network structure and traffic flow.

- Dijkstra's Algorithm: In airline systems, the shortest path between 2 cities is determined based on connectivity and not geography. In graph data science, moving across these connected routes are called multi-hop traversals. We can apply Dijkstra's Algorithm in our project to discover new routes and apply optimal travel by discovering the shortest paths. This algorithm discovers the direct path among nodes and uses edges' weights to predict a path that reduces overall distance amongst the origin node and the rest of the nodes. [3]

6.4 Role of Metadata in Graph Data Science

Metadata plays an important role in graph data science by providing contextual context, structure and governance for the data they contain [7]. This adds layers of descriptions to the nodes and relationships of the graph database, enabling users and systems to understand how, why and when the data was created, modified or connected. This metadata might include timestamps to track when a node was updated or a relationship was added. Without metadata, the graph would lack the required structural framework to reliably interpret or validate its contents.

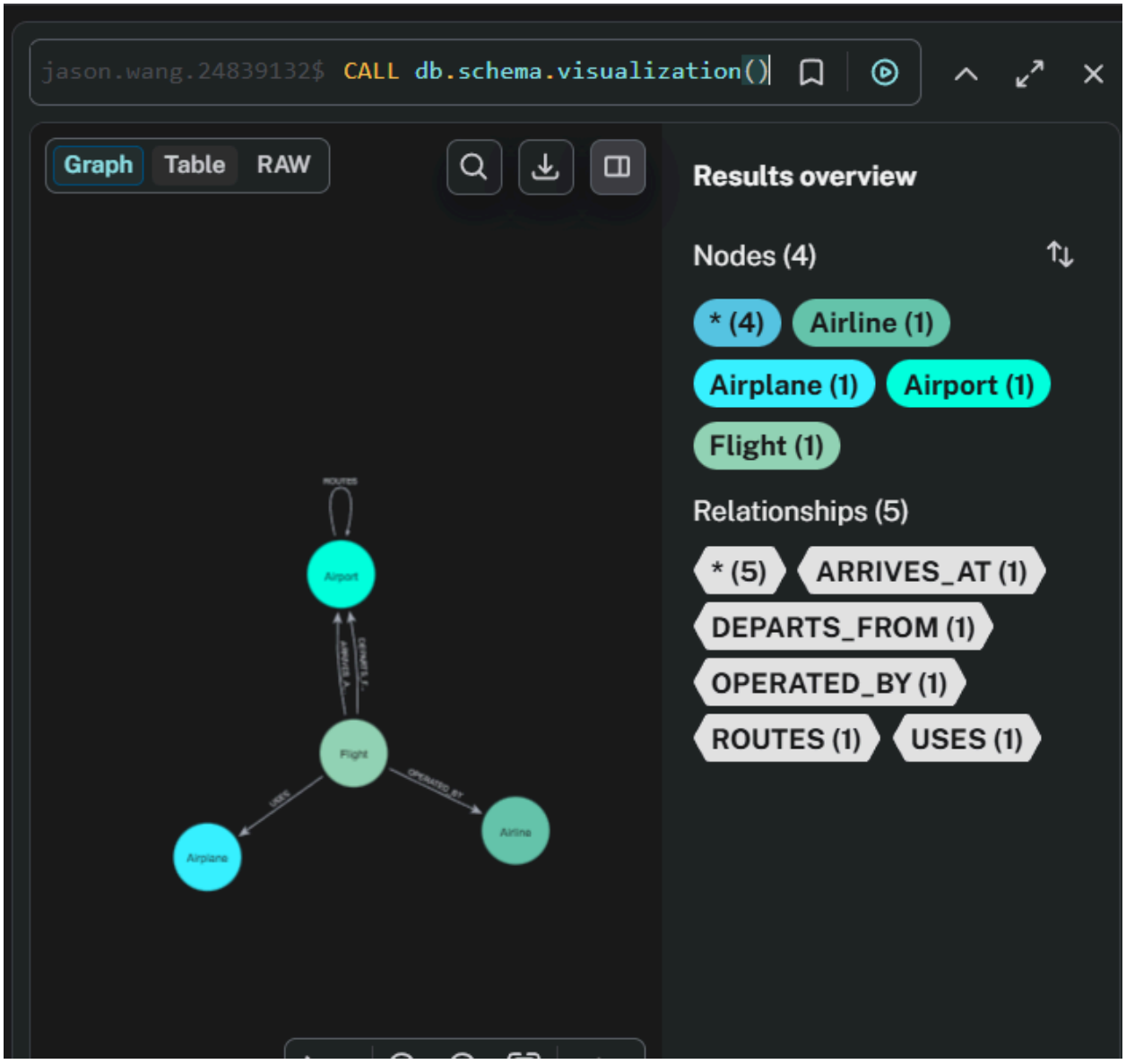
One benefit of using metadata when designing a graph database schema is it unifies metadata by breaking down siloes and connecting all data systems [8]. Another is a greater trust in data, metadata increases trust in data by creating a platform for enabling data asset ownership, certification and context across functions [8]. Metadata also plays a key role in data governance and compliance by enabling the integration of business and technical metadata within a unified knowledge graph[9].

6.5 Leveraging Metadata in Cypher Queries

Metadata in the graph database reveals some important structural information about the schema design. This includes the labels of the nodes, the relationships used to connect the nodes and the properties that define these nodes. Data engineers and scientists can leverage this metadata to write faster and more efficient cypher queries in several ways.

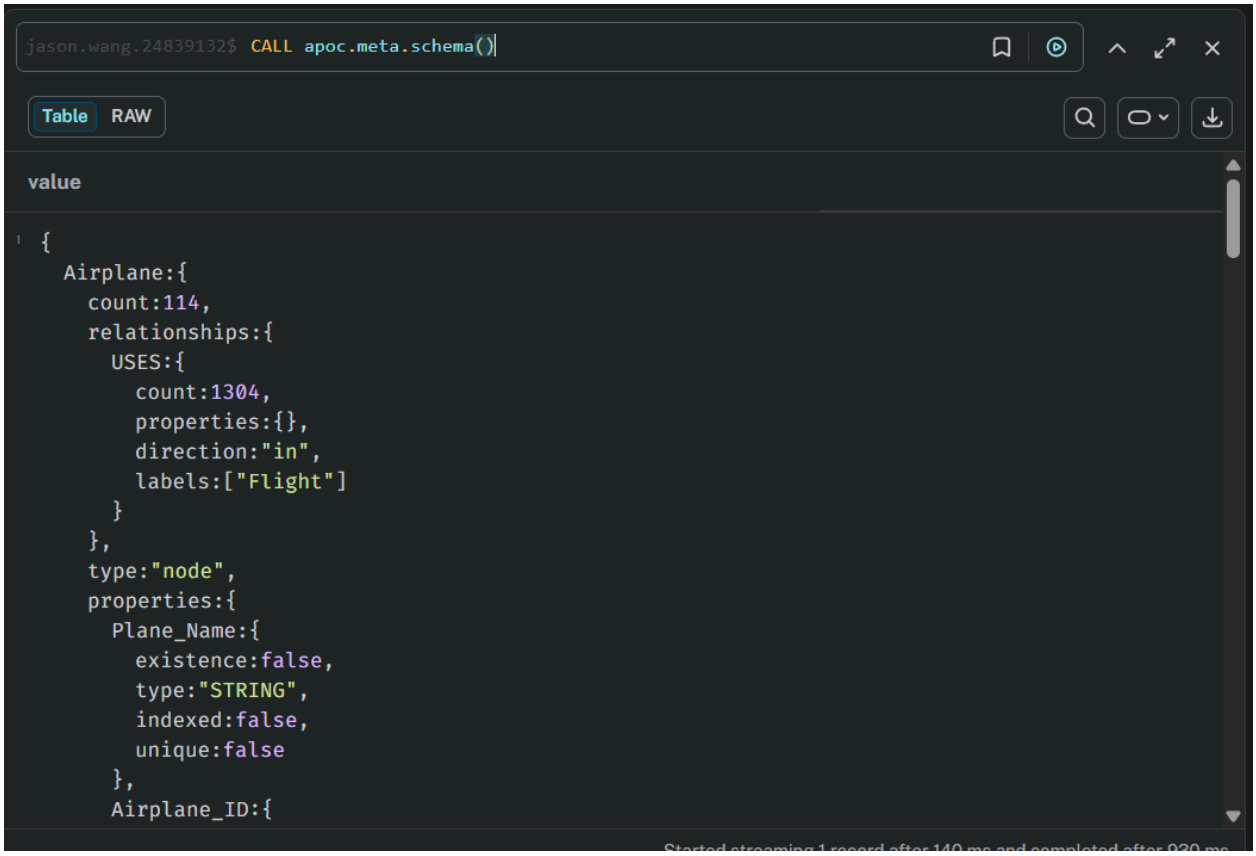
One way to leverage metadata is by using db.schema.visualization() to visualize the graph database's overall structure, this returns all label nodes and relationships and how they are connected, allowing users to understand the graph structure without the need of querying the actual data[9]. In the context of this project, this can be used to confirm if all the nodes and

relationships are correctly connected with each other, such as the Flight node is connected to the Airplane node with USES relationships. This can help developers from writing incorrect or unnecessary traversal patterns.



Another way is by using apoc.meta.schema() to check the property keys and datatypes defined for the nodes and relationships[10]. For example, in the Airport node, this command can be used to show that it has 4 properties, Airport_ID, Airport_Name, Airport_City, and Airport_Country, once understood users can use this to write more targeted queries, applying it to the correct property name and avoiding runtime errors that may occur due to incorrect or non existing properties. Developers can also write more targeted and accurate queries using the correct property names and appropriate data types, which reduces runtime errors caused by incorrect properties and improves query readability. Also, understanding the schema metadata

helps developers optimise query design by filtering relevant properties and avoiding unnecessary traversals across unrelated nodes and relationships.



7. References

[1] U. Kang, C. E. Tsourakakis, and C. Faloutsos, "Pegasus: a peta-scale graph mining system implementation and observations," in *Proc. 9th IEEE Int. Conf. Data Mining (ICDM'09)*, New York, NY, USA, 2009, pp. 229–238.

[2] TigerGraph, "Graph data science." Accessed: May 3, 2026. [Online]. Available: https://www.tigergraph.com/glossary/graph-data-science-2/

[3] P. Raj et al., Eds., *Demystifying Graph Data Science: Graph Algorithms, Analytics Methods, Platforms, Databases, and Use Cases*. Institution of Engineering and Technology, 2022.

[4] InetSoft, "What is graph data science?" Accessed: May 3, 2026. [Online]. Available: https://www.inetsoft.com/info/what-is-graph-data-science/

[5] Neo4j, Inc., "Louvain," Neo4j Graph Data Science Documentation. Accessed: May 3, 2026. [Online]. Available: https://neo4j.com/docs/graph-data-science/current/algorithms/louvain/

[6] Neo4j, Inc., "Link prediction," Neo4j Graph Data Science Documentation. Accessed: May 3, 2026. [Online]. Available: https://neo4j.com/docs/graph-data-science/current/machine-learning/linkprediction-pipelines/link-prediction/

[7] Milvus, "What is the role of metadata in knowledge graphs?" Accessed: May 3, 2026. [Online]. Available: https://milvus.io/ai-quick-reference/what-is-the-role-of-metadata-in-knowledge-graphs

[8] Atlan, "Metadata knowledge graph: What are the core benefits of a metadata knowledge graph?" Accessed: May 3, 2026. [Online]. Available: https://atlan.com/know/metadata-knowledge-graph/#what-are-the-core-benefits-of-a-metadata-knowledge-graph

[9] Neo4j, Inc., "db.schema.visualization — Procedures," Neo4j Operations Manual. Accessed: May 3, 2026. [Online]. Available: https://neo4j.com/docs/operations-manual/current/procedures/#procedure_db_schema_visualization

[10] Neo4j, Inc., "apoc.meta.schema," APOC Core Documentation. Accessed: May 3, 2026. [Online]. Available: https://neo4j.com/docs/apoc/current/overview/apoc.meta/apoc.meta.schema/

8. Acknowledgement of AI Assistance

In the preparation of this report, generative AI tools were used to assist in the following areas:

- Providing inspiration and clarification for the concepts of Graph Data Science and Metadata in graph based models.
- Providing insights and suggestions on the property graph model design, specifically suggesting to use a flight node to represent each record from the original dataset.
- Providing suggestions on the ETL process, particularly the expanding of the Plane Name variable.
- Provide clarification and better understanding of Cypher query commands.
- Help compile all the references into IEE citation style.

All AI-generated suggestions were critically evaluated and selectively incorporated based on the relevance and correctness. The design, analysis, implementation and writing present in the report are entirely my work.